Rockchip Developer Guide PCIE EP标准卡APIs

文件标识：RK-KF-YF-C52

发布版本：V3.4.0

日　　期：2026-03-20

文件密级：□绝密　□秘密　□内部资料　■公开

免责声明

本文档按“现状”提供，瑞芯微电子股份有限公司（“本公司”，下同）不对本文档的任何陈述、信息和内容的准确性、可靠性、完整性、适销性、特定目的性和非侵权性提供任何明示或暗示的声明或保证。本文档仅作为使用指导的参考。

由于产品版本升级或其他原因，本文档将可能在未经任何通知的情况下，不定期进行更新或修改。

商标声明

“Rockchip”、“瑞芯微”、“瑞芯”均为本公司的注册商标，归本公司所有。

本文档可能提及的其他所有注册商标或商标，由其各自拥有者所有。

版权所有 © 2026 瑞芯微电子股份有限公司

超越合理使用范畴，非经本公司书面许可，任何单位和个人不得擅自摘抄、复制本文档内容的部分或全部，并不得以任何形式传播。

瑞芯微电子股份有限公司

Rockchip Electronics Co., Ltd.

地址：　福建省福州市铜盘路软件园A区18号

网址：　www.rock-chips.com

客户服务电话：+86-4007-700-590

客户服务传真：+86-591-83951833

客户服务邮箱：fae@rock-chips.com

前言

概述

本文档对RK PCIE 标准EP开发包使用进行说明，以及对相关API做介绍。

产品版本

芯片名称	内核版本
所有支持RK PCIe EP的主芯片	Linux 6.1、5.10、4.19、RT-Thread v5.1.x、v4.1.x

说明：

- 仅在 RK PCIe EP 侧实现 RT-Thread 支持

读者对象

本文档（本指南）主要适用于以下工程师：

技术支持工程师

软件开发工程师

修订记录

版本号	作者	修改日期	修改说明
V1.0.0	yp.xiao，Kever Yang，Jon Lin	2023-03-07	初始版本
V1.1.0	yp.xiao	2023-05-22	更新SDK API以及部分配置说明
V1.2.0	yp.xiao	2023-06-15	添加EP卡固件编译、烧录、OTA升级说明 添加应用异常说明 更新`struct pcie_dev_attr_st`结构体，支持超时配置、PCIE速率获取、RC DMA配置。
v1.3.0	Jon Lin	2023-10-07	增加pcie.bin说明
v1.4.0	yp.xiao, Jon Lin	2023-10-23	更新SDK API 添加EP验收标准章节 更新双存储说明 更新PCIE BIN说明

版本号	作者	修改日期	修改说明
v1.5.0	Jon Lin	2024-05-23	更新PHY Mode设置
v1.6.0	Jon Lin	2024-09-18	添加用户态测试Demo - pcie_speed_test性能参考、增加BAR1/5支持
v1.7.0	yp.xiao, Jon Lin	2025-01-24	chips_connect开发包增加OSAL、兼容Windows版本
v1.8.0	Jon Lin	2025-02-12	优化EP启动方案说明、修改排版问题
v2.0.0	Jon Lin	2025-03-09	增加RK芯片互联独立供电方案说明
v3.0.0	Jon Lin	2025-05-14	调整须知为EP卡规格、优化EP Boot说明、增加数据传输原理说明、增加用户层多应用说明
v3.1.0	Jon Lin	2025-07-12	添加 RK1820
v3.2.0	Jon Lin	2025-08-11	独立输出Chips Connect APIs文档
v3.3.0	Jon Lin	2025-08-15	添加 err_event，pcie_is_alive 和 hardware_reset 接口
v3.4.0	yp.xiao	2026-03-20	添加日志设置、buff替换、固件上传等新接口；更新接口参数类型

目录

Rockchip Developer Guide PCIE EP标准卡APIs

1. Chips Connect API参考
    1.1 PCIE Linux RC
        1.1.1 rk_pcie_get_pci_bus_addresses
        1.1.2 rk_pcie_device_init
        1.1.3 rk_pcie_device_deinit
        1.1.4 rk_pcie_task_create
        1.1.5 rk_pcie_task_destroy
        1.1.6 rk_pcie_boot_init
        1.1.7 rk_pcie_boot_deinit
        1.1.8 rk_pcie_boot_download_firmware
        1.1.9 rk_pcie_boot_run
        1.1.10 rk_pcie_get_ep_info
        1.1.11 rk_pcie_get_user_cmd
        1.1.12 rk_pcie_get_buff
        1.1.13 rk_pcie_send_buff
        1.1.14 rk_pcie_get_bar1_user_buffer
        1.1.15 rk_pcie_get_bar5_user_buffer
        1.1.16 rk_pcie_set_bar_inbound_cpu_addr
        1.1.17 rk_pcie_release_buff
        1.1.18 rk_pcie_get_buff_max_size
        1.1.19 rk_pcie_get_ep_mode
        1.1.20 rk_pcie_rescan_devices
        1.1.21 rk_pcie_send_msg_to_ep_bus
        1.1.22 rk_pcie_get_msg_for_task_id
        1.1.23 rk_pcie_get_ep_obj_info
        1.1.24 rk_pcie_get_ep_err_event_count
        1.1.25 rk_pcie_is_alive
        1.1.26 rk_pcie_set_log_level
        1.1.27 rk_pcie_set_log_cb
        1.1.28 rk_pcie_device_hardware_reset
        1.1.29 rk_pcie_device_hardware_reset_sync
        1.1.30 rk_pcie_is_device_hardware_reset
        1.1.31 rk_pcie_boot_upload_firmware
        1.1.32 rk_pcie_get_boot_state
        1.1.33 rk_pcie_set_fw_info
        1.1.34 rk_pcie_send_merge_msg_to_ep_bus
        1.1.35 rk_pcie_replace_buff_all
        1.1.36 rk_pcie_replace_buff
        1.1.37 数据类型
            1.1.37.1 RK_PCIE_HANDLES
            1.1.37.2 log_callback
            1.1.37.3 EBuff_Type
            1.1.37.4 ETask_Msg_Type
            1.1.37.5 pcie_buff_node
            1.1.37.6 pcie_dev_bus
            1.1.37.7 pcie_dev_mode
            1.1.37.8 pcie_dev_attr_st
            1.1.37.9 pcie_task_attr_st
            1.1.37.10 pcie_task_msg_st
            1.1.37.11 pcie_ep_obj_info_for_user
            1.1.37.12 pcie_err_event_count
            1.1.37.13 pcie_replace_buff_info_st
            1.1.37.14 pcie_replace_buff_info_list_st
    1.2 PCIE Windows EP

1.2.1 数据类型
1.2.1.1 RK_PCIE_TASK_ATTR
1.2.1.2 RK_PCIE_DESTROY_TASK
1.2.1.3 RK_PCIE_IOCTL_STATUS
1.2.1.4 RK_PCIE_TASK_MSG
1.2.1.5 RK_PCIE_BUFF_NODE
1.2.1.6 RK_PCIE_DEV_MODE
1.2.1.7 RK_PCIE_TASK_INFO
1.2.1.8 RK_PCIE_DOWNLOAD_FIRMWARE
1.3 PCIE EP
1.3.1 rk_pcie_set_log_level
1.3.2 rk_pcie_set_log_cb
1.3.3 rk_pcie_device_init
1.3.4 rk_pcie_device_deinit
1.3.5 rk_pcie_task_create
1.3.6 rk_pcie_task_destroy
1.3.7 rk_pcie_set_ep_info
1.3.8 rk_pcie_get_user_cmd
1.3.9 rk_pcie_get_bar1_user_buffer
1.3.10 rk_pcie_get_bar5_user_buffer
1.3.11 rk_pcie_set_bar_inbound_cpu_addr
1.3.12 rk_pcie_get_buff
1.3.13 rk_pcie_send_buff
1.3.14 rk_pcie_release_buff
1.3.15 rk_pcie_get_buff_max_size
1.3.16 rk_pcie_get_msg_for_bus
1.3.17 rk_pcie_send_msg_to_rc_task
1.3.18 rk_pcie_check_buff_avail
1.3.19 rk_pcie_get_err_event_count
1.3.20 rk_pcie_is_alive
1.3.21 rk_pcie_replace_buff_all
1.3.22 rk_pcie_replace_buff
1.3.23 数据类型
1.3.23.1 EBuff_Type
1.3.23.2 ETask_Msg_Type
1.3.23.3 pcie_buff_node
1.3.23.4 pcie_task_msg_st
1.4 错误码

1. Chips Connect API参考

1.1 PCIE Linux RC

1.1.1 rk_pcie_get_pci_bus_addresses

【描述】

获取系统中所有匹配的PCI设备总线地址。

【语法】

```
int rk_pcie_get_pci_bus_addresses(int vendor_id, int device_id, struct
pcie_dev_bus *pci_bus)
```

【参数】

参数名称	描述	输入/输出
vendor_id	PCIE 厂商标识	输入
device_id	PCIE 产品标识	输入
pci_bus	pci设备总线地址结构体	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

【举例】

请参见rk_pcie_device_init的举例。

1.1.2 rk_pcie_device_init

【描述】

初始化pcie设备。

【语法】

```
RK_PCIE_HANDLES rk_pcie_device_init(struct pcie_dev_attr_st *dev)
```

【参数】

参数名称	描述	输入/输出
dev	PCIE设备配置结构体	输入

【返回值】

返回值	描述
非NULL	成功
NULL	失败

【举例】

```
#define VENDOR_ID                   0x1d87
#define DEVICE_ID                   0x356a

struct pcie_dev_bus dev_bus;
struct pcie_dev_attr_st dev_attr;
RK_PCIE_HANDLES rk_handle;
int ret;

ret = rk_pcie_get_pci_bus_addresses(VENDOR_ID, DEVICE_ID, &dev_bus);
if (ret != 0) {
    printf("get pci bus address fail\n");
    return -1;
}

printf("ep dev max num : %d\n", dev_bus.dev_max_num);
if (dev_bus.dev_max_num == 0) {
    printf("no ep device, exit ... \n");
} else {
    //初始化PCIE设备
    dev_attr.using_rc_dma = 0; //关闭RC DMA，使用EP DMA传输
    dev_attr.rc_dma_chn = 0;
    dev_attr.vendor_id = VENDOR_ID;
    dev_attr.device_id = DEVICE_ID;
    dev_attr.use_dma_only_in_rc = 0;
    //选择第一个匹配设备
    memcpy(dev_attr.pci_bus_address, dev_bus.pci_bus_address[0],
strlen(dev_bus.pci_bus_address[0]));

    //初始化EP设备
    rk_handle = rk_pcie_device_init(&dev_attr);
    if (rk_handle == NULL) {
        printf("pcie device init fail:%s \n", dev_attr.pci_bus_address);
        return -1;
    }

    //反初始化pcie设备，释放资源
    ret = rk_pcie_device_deinit(rk_handle, 0);
    if (ret != 0) {
        printf("pcie deinit device fail....ret = %d\n", ret);
        return -1;
    }
}
```

【相关主题】

rk_pcie_device_deinit

1.1.3 rk_pcie_device_deinit

【描述】

反初始化pcie设备，释放资源。

【语法】

```
int rk_pcie_device_deinit(RK_PCIE_HANDLES handles, int en_remove)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
en_remove	释放资源并移除该设备	输入

【返回值】

返回值	描述
0	成功
-1	句柄无效
-2	设备未初始化
其他	异常，请参见 错误码

【举例】

请参见 rk_pcie_device_init 的举例。

【相关主题】

rk_pcie_device_init

1.1.4 rk_pcie_task_create

【描述】

根据属性创建task, 申请对应内存、BUFF管理等。该接口会阻塞等待EP设备调用task创建接口。

【语法】

```
int rk_pcie_task_create(RK_PCIE_HANDLES handles, struct pcie_task_attr_st *task)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task	PCIE Task配置结构体	输入

【返回值】

返回值	描述
大于0	正常，task_id
小于0	异常，请参见错误码

【举例】

```
//选择第一个匹配设备
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("pcie device init fail:%s \n", rc_ctx.dev_attr.pci_bus_address);
    return -1;
}

rc_ctx.task_attr.send_buff_num = 5; //发送BUF个数
rc_ctx.task_attr.send_buff_size = 1048576;    //发送BUF大小
rc_ctx.task_attr.recv_buff_num = 5; //接收BUF个数
rc_ctx.task_attr.recv_buff_size = 8294408 ;   //接收BUF大小
rc_ctx.task_attr.wait_timeout_ms = -1;   //阻塞等待
rc_ctx.task_id = rk_pcie_task_create(rc_ctx.rk_handle, &rc_ctx.task_attr);
if (rc_ctx.task_id < 0) {
    printf("pcie init create task fail....\n");
    return -1;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("ep ap init, dev mode: %d, submode: %d\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);

ret = rk_pcie_task_destroy(rc_ctx.rk_handle, rc_ctx.task_id);
if (ret != 0) {
    printf("pcie destroy task fail....ret = %d\n", ret);
    return -1;
}
```

【相关主题】

rk_pcie_task_destroy

【注意】

该接口依赖`rk_pcie_device_init`，需要先调用`rk_pcie_device_init`初始化设备才能进行task初始化。

1.1.5 rk_pcie_task_destroy

【描述】

反初始化Task，释放Task对应的内存、BUFF管理等资源。

【语法】

```
int rk_pcie_task_destroy(RK_PCIE_HANDLES handles, int task_id)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

请参见 rk_pcie_task_create 的举例。

【相关主题】

rk_pcie_task_create

1.1.6 rk_pcie_boot_init

【描述】

初始化PCIE BOOT。

【语法】

```
int rk_pcie_boot_init(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

```
//初始化PCIE设备
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("device init fail \n");
    return -1;
}

//初始化PCIE Boot
ret = rk_pcie_boot_init(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie boot init ep fail....ret = %d\n", ret);
    return -1;
}

// pcie boot
rk_pcie_boot_download_firmware_list(&rc_ctx, &pcie_boot_info, &pcie_boot_ini);

// ep boot
rk_ep_boot_download_firmware_list(&rc_ctx, &ep_boot_info, &ep_boot_ini);

//释放PCIE Boot资源
ret = rk_pcie_boot_deinit(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie boot deinit ep fail....ret = %d\n", ret);
    return -1;
}

printf("download ep end\n");
```

【相关主题】

rk_pcie_boot_deinit

【注意】

该接口依赖`rk_pcie_device_init`，需要先调用`rk_pcie_device_init`初始化设备才能进行boot初始化。

1.1.7 rk_pcie_boot_deinit

【描述】

反初始化PCIE BOOT。

【语法】

```
int rk_pcie_boot_deinit(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

rk_pcie_boot_init

【相关主题】

rk_pcie_boot_init

1.1.8 rk_pcie_boot_download_firmware

【描述】

下载PCIE BOOT固件。

【语法】

```
int rk_pcie_boot_download_firmware(RK_PCIE_HANDLES handles, unsigned char
*firmware, unsigned long long firmware_size, unsigned long long
remote_load_addr)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
firmware	固件数据	输入
firmware_size	固件大小	输入
remote_load_addr	远程固件加载地址	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

```
int rk_ep_download_firmware(struct rc_context_st *rc_ctx, char *firmware,
unsigned long load_addr)
{
    int fd;
    unsigned long len;
    unsigned long size = 1024 * 1024;
```

```
    void *buffer = malloc(size);
    int ret;
    fd = open(firmware, O_RDONLY);
    if (fd == -1) {
        printf(" open %s file error, ret = %d \n", firmware, fd);
        return -1;
    }

    while ((len = read(fd, buffer, size)) > 0) {
        ret = rk_pcie_boot_download_firmware(rc_ctx->rk_handle, buffer, len,
load_addr);
        if (ret != 0) {
            return -1;
        }

        load_addr += len;
    }

    close(fd);
    return 0;
}
```

【相关主题】

rk_ep_download_firmware

1.1.9 rk_pcie_boot_run

【描述】

启动PCIE BOOT固件。

【语法】

```
int rk_pcie_boot_run(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

rk_pcie_boot_init

1.1.10 rk_pcie_get_ep_info

【描述】

获取EP状态信息，最大支持96KB。

【语法】

```
int rk_pcie_get_ep_info(RK_PCIE_HANDLES handles, void *status, size_t
status_size)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
status	void指针，状态结构体由用户自定义	输出
status_size	状态结构体size	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

【举例】

```
struct ep_info_st {
    unsigned int link_status;
    unsigned int temp;
    long long timestamp;
    unsigned char cpu_load;
    unsigned char npu_load;
    unsigned char gpu_load;
};

long long timestamp;
while(1) {
    ret = rk_pcie_get_ep_info(rc_ctx.rk_handle, ep_info, sizeof(struct
ep_info_st));
    if(timestamp == ep_info->timestamp) {
        printf("EP device status is abnormal \n");
        break;
    }
    timestamp = ep_info->timestamp;
}
```

1.1.11 rk_pcie_get_user_cmd

【描述】

获取BAR0特定位置一块内存指针，用于用户自定义命令区。最大支持256KB。

【语法】

```
void *rk_pcie_get_user_cmd(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
非NULL	正常，自定义命令区内存指针
NULL	失败

【举例】

```
struct user_cmd_st {
    unsigned int max_chn;
    unsigned int width;
    unsigned int height;
    unsigned int codecid;
    unsigned int output_id;
    unsigned int init;
};

user_cmd = (struct user_cmd_st *)rk_pcie_get_user_cmd(rc_ctx.rk_handle);
user_cmd->max_chn = rc_ctx.max_chn;
user_cmd->width = rc_ctx.width;
user_cmd->height = rc_ctx.height;
user_cmd->codecid = rc_ctx.codecid;
user_cmd->output_id = 1; //0:RGB888 1:BGRA8888
user_cmd->init = 1;
```

1.1.12 rk_pcie_get_buff

【描述】

获取可用PCIE buff。

【语法】

```
struct pcie_buff_node *rk_pcie_get_buff(RK_PCIE_HANDLES handles, int task_id,
enum EBuff_Type type)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入
type	PCIE Buff类型	输入

【返回值】

返回值	描述
非NULL	可用Buff指针
NULL	无可用Buff指针

【举例】

```
node = rk_pcie_get_buff(ctx->rk_handle, ctx->task_id, E_RECV); //获取可用接收 BUF
if (node) {
    if (check_info(node->vir_addr + node->size - CHECK_SIZE) == 0) { //校验 BUF
数据正确
        pthread_mutex_lock(&ctx->mutex_stream_speed);
        ctx->stream_speed_index++;
        pthread_mutex_unlock(&ctx->mutex_stream_speed);
        memset(node->vir_addr + node->size - CHECK_SIZE, 0, CHECK_SIZE);
    } else {
        printf("check data ==================== error \n");
        for (int i = 0; i < CHECK_SIZE; i++) {
            unsigned char *p = node->vir_addr + node->size - CHECK_SIZE;
            printf("%d ", p[i]);
        }
        printf("\ncheck data ===ssss========node->size = %d======== error \n",
(int)node->size);
    }
    rk_pcie_release_buff(ctx->rk_handle, ctx->task_id, node);  //释放BUF
} else {
    usleep(1000);
}
```

【相关主题】

rk_pcie_release_buff

1.1.13 rk_pcie_send_buff

【描述】

发送PCIE buff，会阻塞至DMA传输完成。

【语法】

```
int rk_pcie_send_buff(RK_PCIE_HANDLES handles, int task_id, struct
pcie_buff_node *pbuff)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入
pbuff	PCIE Buff指针	输入

【返回值】

返回值	描述
0	发送成功，表示数据已经通过DMA传输完成
非0	失败，请参见错误码。

【举例】

```
node = rk_pcie_get_buff(ctx->rk_handle, ctx->task_id, E_SEND); //获取可用的发送BUF
if (node) {
    node->size = buffer_size;
    //操作BUF,写入数据
    memcpy(node->vir_addr, mptr, node->size);
    add_check_info(node->vir_addr + node->size - CHECK_SIZE);
    rk_pcie_send_buff(ctx->rk_handle, ctx->task_id, node); //发送到对端
    index++;
} else {
    usleep(1000);
}
```

1.1.14 rk_pcie_get_bar1_user_buffer

【描述】

获取bar1虚拟映射地址。

【语法】

```
void *rk_pcie_get_bar1_user_buffer(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	所有匹配设备的句柄	输入

【返回值】

返回值	描述
void *	虚拟地址
NULL	失败，请参见错误码。

1.1.15 rk_pcie_get_bar5_user_buffer

【描述】

获取bar5虚拟映射地址。

【语法】

```
void *rk_pcie_get_bar5_user_buffer(RK_PCIE_HANDLES handles)
```

【参数】

参数名称	描述	输入/输出
handles	所有匹配设备的句柄	输入

【返回值】

返回值	描述
void *	虚拟地址
NULL	失败，请参见错误码。

1.1.16 rk_pcie_set_bar_inbound_cpu_addr

【描述】

设置bar inbound映射。

【语法】

```
u_int64_t rk_pcie_set_bar_inbound_cpu_addr(RK_PCIE_HANDLES handles, u_int32_t
atu_index, u_int64_t cpu_address, u_int32_t size)
```

【参数】

参数名称	描述	输入/输出
handles	所有匹配设备的句柄	输入
atu_index	atu index	输入
cpu_address	映射cpu地址	输入
size	映射大小限制	输入

【返回值】

返回值	描述
cpu_address	返回cpu地址

1.1.17 rk_pcie_release_buff

【描述】

释放PCIE buff。

【语法】

```
int rk_pcie_release_buff(RK_PCIE_HANDLES handles, int task_id, struct
pcie_buff_node *pbuff)
```

【参数】

参数名称	描述	输入/输出
handles	所有匹配设备的句柄	输入
task_id	Task ID	输入
pbuff	PCIE Buff指针	输入

【返回值】

返回值	描述
0	释放成功
非0	失败，请参见错误码。

【举例】

rk_pcie_get_buff

1.1.18 rk_pcie_get_buff_max_size

【描述】

获取buff支持的最大size。

【语法】

```
long rk_pcie_get_buff_max_size(RK_PCIE_HANDLES handles, int task_id, enum
EBuff_Type type)
```

【参数】

参数名称	描述	输入/输出
handles	所有匹配设备的句柄	输入
task_id	Task ID	输入
type	PCIE Buff类型	输入

【返回值】

返回值	描述
大于等于0	buff size
小于0	失败，请参见错误码

【举例】

```
long buffer_size = rk_pcie_get_buff_max_size(ctx->rk_handle, ctx->task_id,
E_SEND);
```

1.1.19 rk_pcie_get_ep_mode

【描述】

获取EP设备当前运行模式，支持从loader、kernel、application等模式下状态获取。模式信息可以在rk_pcie_rc.h获取

【语法】

```
int rk_pcie_get_ep_mode(RK_PCIE_HANDLES handles, struct pcie_dev_mode *devmode)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
devmode	设备模式结构体	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
struct rc_context_st {
    RK_PCIE_HANDLES rk_handle;
    struct pcie_task_attr_st dev_attr;
```

```
    struct pcie_dev_mode devmode;
}rc_ctx;

//选择第一个匹配设备初始化
rc_ctx.rk_handle = rk_pcie_device_init(&rc_ctx.dev_attr);
if (rc_ctx.rk_handle == NULL) {
    printf("device init fail \n");
    usleep(1000000);
    continue;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("111 ep device init, dev mode: 0x%.2x, submode: 0x%.2x\n",
rc_ctx.devmode.mode, rc_ctx.devmode.submode);
```

1.1.20 rk_pcie_rescan_devices

【描述】

重新扫描系统PCI设备，EP设备异常重启后恢复需要remove设备再rescan设备。

【语法】

```
void rk_pcie_rescan_devices(void)
```

【参数】

无

【返回值】

无

【举例】

无

1.1.21 rk_pcie_send_msg_to_ep_bus

【描述】

发送消息到EP端，单个消息最大支持1504Byte

【语法】

```
int rk_pcie_send_msg_to_ep_bus(RK_PCIE_HANDLES handles, int task_id, void *data,
size_t size)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入
data	消息指针	输入
size	消息长度	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
rc_ctx.task_attr.send_buff_num = 5;
rc_ctx.task_attr.send_buff_size = 5 * 1024 * 1024;
rc_ctx.task_attr.recv_buff_num = 5;
rc_ctx.task_attr.recv_buff_size = 5 * 1024 * 1024;
rc_ctx.task_attr.wait_timeout_ms = -1;
rc_ctx.task_id = rk_pcie_task_create(rc_ctx.rk_handle, &rc_ctx.task_attr);
if (rc_ctx.task_id < 0) {
    printf("pcie init create task fail....\n");
    return -1;
}

rk_pcie_get_ep_mode(rc_ctx.rk_handle, &rc_ctx.devmode);
printf("ep ap init, dev mode: 0x%.2x, submode: 0x%.2x\n", rc_ctx.devmode.mode,
rc_ctx.devmode.submode);

rc_ctx.user_cmd.cmd = EP_TEST_STATUS_RUN_APP;
rk_pcie_send_msg_to_ep_bus(rc_ctx.rk_handle, rc_ctx.task_id, (void
*)&rc_ctx.user_cmd, sizeof(struct task_user_cmd_st));
```

1.1.22 rk_pcie_get_msg_for_task_id

【描述】

通过Task id获取EP消息

【语法】

```
int rk_pcie_get_msg_for_task_id(RK_PCIE_HANDLES handles, struct pcie_task_msg_st
*msg, int task_id, int timeout_ms)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
msg	消息结构体指针	输出
task_id	任务ID	输入
timeout_ms	超时时间，-1为阻塞模式(不推荐，会一直在内核等待)，建议100ms，单位ms	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
unsigned char cmd;
struct pcie_task_msg_st msg;
ret = rk_pcie_get_msg_for_task_id(rc_ctx.rk_handle, &msg, rc_ctx.task_id, 100);
if (ret == RK_PCIE_OK) {
    cmd = msg.data[0];
    printf("recv ep msg: %d \n", cmd);
    if (cmd == 10) {
        break;
    }
}
```

1.1.23 rk_pcie_get_ep_obj_info

【描述】

获取EP设备基础信息

【语法】

```
int rk_pcie_get_ep_obj_info(RK_PCIE_HANDLES handles, struct
pcie_ep_obj_info_for_user *info)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
info	基础信息缓存buffer	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
struct pcie_ep_obj_info_for_user info;
ret = rk_pcie_get_ep_obj_info(rc_ctx.rk_handle, &info);
if (ret == RK_PCIE_OK) {
    return ret;
}
```

1.1.24 rk_pcie_get_ep_err_event_count

【描述】

获取PCIe EP异常事件计数

【语法】

```
int rk_pcie_get_ep_err_event_count(RK_PCIE_HANDLES handles, struct
pcie_err_event_count *info)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
info	PCIe异常事件计数	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
struct pcie_err_event_count info;
ret = rk_pcie_get_ep_err_event_count(rc_ctx.rk_handle, &info);
if (ret == RK_PCIE_OK) {
    return ret;
}
```

1.1.25 rk_pcie_is_alive

【描述】

检查PCIe 链路是否正常

【语法】

```
int rk_pcie_is_alive(RK_PCIE_HANDLES handles);
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
cc_true	链路正常
cc_false	链路异常

【举例】

```
ret = rk_pcie_is_alive(rc_ctx.rk_handle);
if (!ret) {
    printf("PCIe link down\n");
    return ret;
}
```

1.1.26 rk_pcie_set_log_level

【描述】

设置日志打印等级，默认是debug。

【语法】

```
int rk_pcie_set_log_level(int level)
```

【参数】

参数名称	描述	输入/输出
level	日志等级，可选：6(verbose)、5(debug)、4(info)、3(warn)、2(error)、1(fatal)	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

1.1.27 rk_pcie_set_log_cb

【描述】

设置日志回调，可以把日志从默认console打印回调给调用者。

【语法】

```
int rk_pcie_set_log_cb(log_callback cb)
```

【参数】

参数名称	描述	输入/输出
cb	日志回调函数指针，类型为`int (*log_callback)(char *log)`	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

1.1.28 rk_pcie_device_hardware_reset

【描述】

内核驱动定制接口硬件复位控制器，需设备驱动实现PCIE_EP_RESET_CTRL ioctl操作，过程中会控制PERST#信号硬件复位EP

【语法】

```
int rk_pcie_device_hardware_reset(RK_PCIE_HANDLES handles);
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
0	复位成功
非0	复位失败或不支持复位，请参见错误码

【举例】

复位控制器前需停止所有PCIe传输，例如：

```
printf("Reset beginning...\n");
/* Reset Progress: Stop all transmission */
rc_ctx.recv_ep_exit = 1;
rc_ctx.send_ep_exit = 1;
for (int i = 0; i < THREADS_MAX; i++) {
    pthread_join(rc_ctx.thr_recv_ep[i], NULL);
    pthread_join(rc_ctx.thr_send_ep[i], NULL);
}
usleep(500000);

/* Reset Progress: Destroy task */
ret = rk_pcie_task_destroy(rc_ctx.rk_handle, rc_ctx.task_id);
if (ret != 0) {
    printf("pcie destroy task fail....ret = %d\n", ret);
    return -1;
}
usleep(100000);

/* Reset Progress: Hardware reset */
ret = rk_pcie_device_hardware_reset(rc_ctx.rk_handle);
if (ret != 0) {
    printf("pcie device hardware reset fail....ret = %d\n", ret);
}

/* Reset Progress: Wait for link up */
usleep(1000000);

/* Reset Progress: Deinit device */
ret = rk_pcie_device_deinit(rc_ctx.rk_handle, 1);
if (ret != 0) {
    printf("pcie deinit device fail....ret = %d\n", ret);
    return -1;
}

/* Reset Progress: Rescan PCI */
rk_pcie_rescan_devices();
printf("Reset done\n");

return 0;
```

1.1.29 rk_pcie_device_hardware_reset_sync

【描述】

内核驱动定制接口硬件复位控制器，带同步等待功能。复位前会等待EP端确认，过程中会控制PERST#信号硬件复位EP。

【语法】

```
int rk_pcie_device_hardware_reset_sync(RK_PCIE_HANDLES handles, int task_id, int timeout_ms);
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入
timeout_ms	超时时间，单位毫秒	输入

【返回值】

返回值	描述
0	复位成功
非0	复位失败或不支持复位，请参见错误码

1.1.30 rk_pcie_is_device_hardware_reset

【描述】

检查PCIe链路是否正在复位。

【语法】

```
int rk_pcie_is_device_hardware_reset(RK_PCIE_HANDLES handles);
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入

【返回值】

返回值	描述
cc_true	正在复位中
cc_false	未在复位状态

1.1.31 rk_pcie_boot_upload_firmware

【描述】

从EP端上传固件数据。

【语法】

```
int rk_pcie_boot_upload_firmware(RK_PCIE_HANDLES handles, unsigned char
*firmware, unsigned long long firmware_size, unsigned long long
remote_load_addr)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
firmware	固件数据缓冲区	输出
firmware_size	固件大小	输入
remote_load_addr	远程固件读取地址	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

1.1.32 rk_pcie_get_boot_state

【描述】

获取EP端的启动状态。

【语法】

```
int rk_pcie_get_boot_state(RK_PCIE_HANDLES handles, unsigned int off)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
off	状态寄存器偏移量	输入

【返回值】

返回值	描述
>=0	启动状态值
<0	失败，请参见错误码。

1.1.33 rk_pcie_set_fw_info

【描述】

设置固件信息到EP端。

【语法】

```
int rk_pcie_set_fw_info(RK_PCIE_HANDLES handles, unsigned char *buf, unsigned
int off, unsigned int size)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
buf	固件信息数据	输入
off	写入偏移量	输入
size	数据大小	输入

【返回值】

返回值	描述
0	正常
非0	失败，请参见错误码。

1.1.34 rk_pcie_send_merge_msg_to_ep_bus

【描述】

发送聚合消息到EP端，用于发送需要聚合处理的数据。

【语法】

```
int rk_pcie_send_merge_msg_to_ep_bus(RK_PCIE_HANDLES handles, int task_id, void
*data, unsigned long long size)
```

【参数】

参数名称	描述	输入/输出
handles	设备句柄	输入
task_id	Task ID	输入
data	消息指针	输入
size	消息长度	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

1.1.35 rk_pcie_replace_buff_all

【描述】

替换所有buff的地址，用于配置用户自定义buff。

【语法】

```
int rk_pcie_replace_buff_all(RK_PCIE_HANDLES handle, int task_id, enum
EBuff_Type type, struct pcie_replace_buff_info_list_st *infos)
```

【参数】

参数名称	描述	输入/输出
handle	设备句柄	输入
task_id	Task ID	输入
type	Buff类型（E_SEND或E_RECV）	输入
infos	替换buff信息列表	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

【相关主题】

rk_pcie_replace_buff
pcie_replace_buff_info_list_st

1.1.36 rk_pcie_replace_buff

【描述】

替换单个buff的地址，用于配置用户自定义buff。

【语法】

```
int rk_pcie_replace_buff(RK_PCIE_HANDLES handle, int task_id, enum EBuff_Type
type, struct pcie_buff_node *pbuff, pcie_replace_buff_info_st *info)
```

【参数】

参数名称	描述	输入/输出
handle	设备句柄	输入
task_id	Task ID	输入
type	Buff类型（E_SEND或E_RECV）	输入
pbuff	原buff节点指针	输入
info	新buff信息	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

【相关主题】

rk_pcie_replace_buff_all
pcie_replace_buff_info_st

1.1.37 数据类型

PCIE RC相关数据类型、数据结构定义如下：

- RK_PCIE_HANDLES：定义设备的句柄。
- log_callback：定义日志回调函数类型。
- EBuff_Type：定义PCIE Buff类型。
- ETask_Msg_Type：定义Task消息类型。
- struct pcie_buff_node：定义PCIE buff属性结构体。
- struct pcie_dev_bus：定义PCI设备总线信息。
- struct pcie_dev_mode：定义PCIE EP设备模式。
- struct pcie_dev_attr_st：定义PCIE EP设备属性结构体。
- struct pcie_task_attr_st：定义Task属性结构体。
- struct pcie_task_msg_st：定义Task消息结构体。
- struct pcie_ep_obj_info_for_user：定义PCIE基础信息结构体。
- struct pcie_err_event_count：定义PCIe异常事件信息结构体。

- pcie_replace_buff_info_st：定义替换buff信息结构体。
- struct pcie_replace_buff_info_list_st：定义替换buff信息列表结构体。

1.1.37.1 RK_PCIE_HANDLES

【说明】

定义设备的句柄。

【定义】

```
typedef void* RK_PCIE_HANDLES;
```

1.1.37.2 log_callback

【说明】

定义日志回调函数类型。

【定义】

```
typedef int (*log_callback)(char *log);
```

【成员】

参数名称	描述
log	日志字符串，由调用者传入

1.1.37.3 EBuff_Type

【说明】

定义PCIE Buff类型。

【定义】

```
enum EBuff_Type {
    E_SEND = 0,
    E_RECV,
};
```

【成员】

成员名称	描述
E_SEND	发送buff。
E_RECV	接收buff。

1.1.37.4 ETask_Msg_Type

【说明】

定义PCIE Msg类型。

【定义】

```
enum ETask_Msg_Type {
    E_TASK_MSG_NONE = 0,
    E_TASK_MSG_CREATE,
    E_TASK_MSG_DESTROY,
    E_TASK_MSG_USER,
    E_TASK_MSG_MAX,
};
```

【成员】

成员名称	描述
E_TASK_MSG_NONE	默认值，无意义。
E_TASK_MSG_CREATE	创建任务。
E_TASK_MSG_DESTROY	销毁任务。
E_TASK_MSG_USER	用户自定义消息。
E_TASK_MSG_MAX	最大消息类型。

1.1.37.5 pcie_buff_node

【说明】

定义PCIE buff属性结构体。

【定义】

```
struct pcie_buff_node{
    unsigned char *vir_addr;
    unsigned long long phy_addr;
    unsigned int size;
};
```

【成员】

成员名称	描述
vir_addr	虚拟地址。
phy_addr	物理地址。
size	内存大小。 发送buff：需要用户配置数据大小，不能超过总的buff内存大小。 接收buff：对端发送过来的数据大小。

1.1.37.6 pcie_dev_bus

【说明】

定义PCI设备总线信息。

【定义】

```
struct pcie_dev_bus {
    int dev_max_num;
    char pci_bus_address[PCI_DEV_MAX][PCI_BUS_MAX_NAME];
};
```

【成员】

成员名称	描述
dev_max_num	系统匹配的设备数量。
pci_bus_address	PCI设备的总线地址。

1.1.37.7 pcie_dev_mode

【说明】

定义PCIE EP设备模式。

【定义】

```
struct pcie_dev_mode {
    unsigned short mode;
    unsigned short submode;
};
```

【成员】

成员名称	描述
mode	PCIE EP设备当前模式，取值范围如下： #define RKEP_MODE_BOOTROM 1 #define RKEP_MODE_LOADER 2 #define RKEP_MODE_KERNEL 3 #define RKEP_MODE_FUN0 4 #define RKEP_MODE_ERR 0xffff //设备异常需要重新加载
submode	PCIE EP设备模式对应的状态，取值范围如下： #define RKEP_SMODE_INIT 0 #define RKEP_SMODE_LNKRDY 1 #define RKEP_SMODE_LNKUP 2 #define RKEP_SMODE_FWDLRDY 0x10 #define RKEP_SMODE_FWDLDONE 0x11 #define RKEP_SMODE_APPRDY 0x20 #define RKEP_SMODE_APPDEINIT 0x21 #define RKEP_SMODE_ERR 0xffff

1.1.37.8 pcie_dev_attr_st

【说明】

定义PCIE EP设备属性结构体。

【定义】

```
struct pcie_dev_attr_st {
    int vendor_id;
    int device_id;
    char pci_bus_address[PCI_BUS_MAX_NAME];
    unsigned char using_rc_dma: 1;
    unsigned char rc_dma_chn: 1;
    unsigned char enable_speed: 1;
    unsigned char use_dma_only_in_rc: 1;// 只在RC端调用DMA收发数据
};
```

【成员】

成员名称	描述
vendor_id	EP设备Vendor ID。
device_id	EP设备Device ID。
pci_bus_address	EP设备的总线地址。
using_rc_dma	RK互联场景下，可使能该模式使用RC端DMA增加传输速率。多EP场景下推荐关闭配置，使用EP DMA。
rc_dma_chn	RC DMA支持2个通道，取值0/1。
enable_speed	使能速率统计功能。可通过cat /tmp/pcie_debug*查看当前EP设备总传输速率。
use_dma_only_in_rc	只在RC端调用DMA收发数据

1.1.37.9 pcie_task_attr_st

【说明】

定义PCIE Task属性结构体。

【定义】

```
struct pcie_task_attr_st {
    int wait_timeout_ms;
    unsigned int send_buff_num;
    size_t send_buff_size;
    unsigned int recv_buff_num;
    size_t recv_buff_size;
};
```

【成员】

成员名称	描述
wait_timeout_ms	设置接口超时时间，-1为阻塞模式。
send_buff_num	PCIE发送buff的数量，用于buff轮询。
send_buff_size	发送buff的内存大小。
recv_buff_num	PCIE接收buff的数量，用于buff轮询。
recv_buff_size	接收buff的内存大小。

1.1.37.10 pcie_task_msg_st

【说明】

定义PCIE Task消息结构体。

【定义】

```
struct pcie_task_msg_st {
    enum ETask_Msg_Type type;
    int task_id;
    unsigned int size;
    char lock;
    unsigned char data[TASK_MSG_DATA_SIZE];
};
```

【成员】

成员名称	描述
type	消息类型。
task_id	Task ID。
size	消息长度。
lock	保留字。
data	消息内容。

1.1.37.11 pcie_ep_obj_info_for_user

【说明】

定义PCIE基础信息结构体。

【定义】

```
struct pcie_ep_obj_info_for_user {
    unsigned int magic;
    unsigned int rc_cc_version;
    unsigned int ep_cc_version;
    unsigned int func_drv_version;
    unsigned short vid;
    unsigned short did;
    unsigned short sub_vid;
    unsigned short sub_did;
};
```

【成员】

成员名称	描述
magic	chips connect开发包magic，32bits位宽，默认值为0x524B4550，EP内核驱动赋值
rc_cc_version	RC chips connect开发包版本信息，32bits位宽，递增计数，例如 0x1，对应 V1
ep_cc_version	EP chips connect开发包版本信息，32bits位宽，递增计数，例如 0x1，对应 V1
func_drv_version	PCIe驱动版本信息，32bits位宽，高八位冗余，其余每8Bits代表一个版本信息，例如 0x00030100，对应 V3.1.0
vid	PCI制造商ID，16bits位宽
did	PCI设备ID，16bits位宽
sub_vid	子系统制造商ID，16bits位宽
sub_did	子系统设备ID，16bits位宽

1.1.37.12 pcie_err_event_count

【说明】

定义PCIE异常事件计数信息结构体。

【定义】

```
struct pcie_err_event_count {
    unsigned char ebuf_overflow;
    unsigned char ebuf_under_run;
    unsigned char decode_error;
    unsigned char running_disparity_error;
    unsigned char skp_os_parity_error;
    unsigned char sync_header_error;
    unsigned char ctl_skp_os_parity_error;
    unsigned char detect_ei_infer;
    unsigned char receiver_error;
    unsigned char rx_recovery_request;
    unsigned char n_fts_timeout;
    unsigned char framing_error;
    unsigned char deskew_error;
    unsigned char bad_tlp;
```

```
    unsigned char lrc_error;
    unsigned char bad_dllp;
    unsigned char replay_number_rollover;
    unsigned char replay_timeout;
    unsigned char rx_nak_dllp;
    unsigned char tx_nak_dllp;
    unsigned char retry_tlp;
    unsigned char fc_timeout;
    unsigned char poisoned_tlp;
    unsigned char ecr_error;
    unsigned char unsupported_request;
    unsigned char completer_abort;
    unsigned char completion_timeout;
    /* The location above is hardware information and cannot be changed */
    unsigned char redundant_phase;
};
```

【成员】

成员名称	简称	描述
ebuf_overflow	EBUF Overflow	-
ebuf_under_run	EBUF Under-run	-
decode_error	Decode Error	包解码错误，信号异常
running_disparity_error	Running Disparity Error	极性偏差错误，8bit/10bit编码中0与1数量比例失衡，Gen2信号异常
skp_os_parity_error	SKP OS Parity Error	SKP 序列奇偶校验错误，Gen3信号异常
sync_header_error	SYNC Header Error	异步包头错误，信号问题
ctl_skp_os_parity_error	CTL SKP OS Parity Error	控制的SKP 序列奇偶校验错误，Gen3信号异常
detect_ei_infer	Detect EI Infer	检测到信号串扰
receiver_error	Receiver Error	接收端错误
rx_recovery_request	Rx Recovery Request	RX信号收到外设请求，进入rescoery状态进行信号矫正
n_fts_timeout	N_FTS Timeout	外设的n_fts不达标，L0s回到L0异常进入recovery状态
framing_error	Framing Error	帧格式解码错误，信号异常
deskew_error	Deskew Error	deskew错误，信号异常
bad_tlp	BAD TLP	收到错误TLP包
lrc_error	LCRC Error	link CRC，信号异常
bad_dllp	BAD DLLP	错误的DLLP包，信号异常
replay_number_rollover	Replay Number Rollover	重传包积累太多，replay buffer冲了
replay_timeout	Replay Timeout	误码包重传超时。
rx_nak_dllp	Rx Nak DLLP	收到来自下游的DLLP而拒绝其访问
tx_nak_dllp	Tx Nak DLLP	发往下游的DLLP被拒绝
retry_tlp	Retry TLP	重传的TLP包数量，可以表征误码率
fc_timeout	FC Timeout	更新流控超时
poisoned_tlp	Poisoned TLP	上报一个错误的TLP类型，可能是因为链路原因或RAM的位跳变引起
ecr_error	ECRC Error	end CRC，信号异常

成员名称	简称	描述
unsupported_request	Unsupported Request	收到不支持的请求类型的TLP包，可能是访问外设被拒绝
completer_abort	Completer Abort	设备接收到了RC发来的请求，但是发生了其内部错误，要求终止该访问请求。
completion_timeout	Completion Timeout	读取外设相关资源，TLP返回超时
redundant_phase	Redundant Phase	计数变量，用以判定状态信息在不断更新

1.1.37.13 pcie_replace_buff_info_st

【说明】

定义替换buff信息结构体，与pcie_buff_node结构相同。

【定义】

```
typedef struct pcie_buff_node pcie_replace_buff_info_st;
```

【成员】

请参见pcie_buff_node的成员说明。

1.1.37.14 pcie_replace_buff_info_list_st

【说明】

定义替换buff信息列表结构体，用于批量替换buff地址。

【定义】

```
#define PCIE_REPLACE_BUFF_MAX_NUM       (7)

struct pcie_replace_buff_info_list_st {
    pcie_replace_buff_info_st info[PCIE_REPLACE_BUFF_MAX_NUM];
    unsigned int buff_num;
};
```

【成员】

成员名称	描述
info	替换buff信息数组，最多支持PCIE_REPLACE_BUFF_MAX_NUM个
buff_num	实际替换的buff数量

1.2 PCIE Windows EP

Windows驱动对应用提供不同的ioctl功能：

ioctl	功能说明
IOCTL_RKEP_DOWNLOAD_FIRMWARE	传输固件给EP
IOCTL_RKEP_BOOT_RUN	固件传输完成后，让EP启动
IOCTL_RKEP_CREATE_TASK	创建Task
IOCTL_RKEP_DESTROY_TASK	释放Task
IOCTL_RKEP_SEND_BUF	发送buf到EP
IOCTL_RKEP_GET_BUF	获取EP的buf数据
IOCTL_RKEP_GET_EP_MODE	获取EP当前模式
IOCTL_RKEP_SEND_MSG_TO_EP_BUS	发送消息事件给EP
IOCTL_RKEP_GET_MSG_FOR_TASK_ID	接受EP的消息事件

1.2.1 数据类型

PCIE RC相关数据类型、数据结构定义如下：

- RK_PCIE_TASK_ATTR：定义Task属性结构体。
- RK_PCIE_DESTROY_TASK：释放Task结构体。
- RK_PCIE_IOCTL_STATUS：定义PCIE ioctl调用状态结构体。
- RK_PCIE_TASK_MSG：定义Task消息结构体。
- RK_PCIE_BUFF_NODE：定义PCIE EP BUF结构体。
- RK_PCIE_DEV_MODE：定义PCIE EP设备模式结构体。
- RK_PCIE_TASK_INFO：定义Task信息结构体。
- RK_PCIE_DOWNLOAD_FIRMWARE: 定义固件下载结构体。

1.2.1.1 RK_PCIE_TASK_ATTR

【说明】

定义PCIE Task属性结构体。

【定义】

```
typedef struct _RK_PCIE_TASK_ATTR {
    int WaitTimeoutMs;
    int SendBuffNum;
    int SendBuffSize;
    int RecvBuffNum;
    int RecvBuffSize;
}  RK_PCIE_TASK_ATTR, *PRK_PCIE_TASK_ATTR;
```

【成员】

成员名称	描述
WaitTimeoutMs	设置接口超时时间，不支持阻塞模式。
SendBuffNum	PCIE发送buff的数量，用于buff轮询。
SendBuffSize	发送buff的内存大小。
RecvBuffNum	PCIE接收buff的数量，用于buff轮询。
RecvBuffSize	接收buff的内存大小。

1.2.1.2 RK_PCIE_DESTROY_TASK

【说明】

定义PCIE Task释放结构体。

【定义】

```
typedef struct _RK_PCIE_DESTROY_TASK {
    int TaskID;
}  RK_PCIE_DESTROY_TASK, *PRK_PCIE_DESTROY_TASK;
```

【成员】

成员名称	描述
TaskID	需要释放的Task ID

1.2.1.3 RK_PCIE_IOCTL_STATUS

【说明】

定义PCIE ioctl调用状态结构体。

【定义】

```
typedef struct _RK_PCIE_IOCTL_STATUS {
    int Status;
    char Data[0];
} RK_PCIE_IOCTL_STATUS, *PRK_PCIE_IOCTL_STATUS;
```

【成员】

成员名称	描述
Status	ioctl调用返回值，返回信息可以查询enum _RK_PCIE_RET 枚举列表
Data	ioctl返回数据

1.2.1.4 RK_PCIE_TASK_MSG

【说明】

定义Task消息结构体。

【定义】

```
typedef struct _RK_PCIE_TASK_MSG {
    int TaskID;
    int Size;
    char Data[TASK_MSG_DATA_SIZE];
} RK_PCIE_TASK_MSG, *PRK_PCIE_TASK_MSG;
```

【成员】

成员名称	描述
TaskID	Task ID
Size	消息长度，最大长度为1504Byte
Data	消息内容

1.2.1.5 RK_PCIE_BUFF_NODE

【说明】

定义PCIE EP BUF结构体。

【定义】

```
typedef struct _RK_PCIE_BUFF_NODE {
    int Size;
    int TaskID;
    int TimeOut10Ms;
    char Reserved[52];  // 64 = 52 + 4 + 4 + 4
    char Data[0];
} RK_PCIE_BUFF_NODE, *PRK_PCIE_BUFF_NODE;
```

【成员】

成员名称	描述
Size	数据长度
TaskID	Task ID
TimeOut10Ms	设置接口超时时间，不支持阻塞模式。最小单位10ms
Reserved	保留大小
Data	数据

1.2.1.6 RK_PCIE_DEV_MODE

【说明】

定义PCIE EP设备模式结构体。

【定义】

```
typedef struct _RK_PCIE_DEV_MODE {
    unsigned short Mode;
    unsigned short SubMode;
} RK_PCIE_DEV_MODE, *PRK_PCIE_DEV_MODE;
```

【成员】

成员名称	描述
Mode	PCIE EP设备当前模式，取值范围如下： #define RKEP_MODE_BOOTROM 1 #define RKEP_MODE_LOADER 2 #define RKEP_MODE_KERNEL 3 #define RKEP_MODE_FUN0 4 #define RKEP_MODE_ERR 0xffff //设备异常需要重新加载
SubMode	PCIE EP设备模式对应的状态，取值范围如下： #define RKEP_SMODE_INIT 0 #define RKEP_SMODE_LNKRDY 1 #define RKEP_SMODE_LNKUP 2 #define RKEP_SMODE_FWDLRDY 0x10 #define RKEP_SMODE_FWDLDONE 0x11 #define RKEP_SMODE_APPRDY 0x20 #define RKEP_SMODE_APPDEINIT 0x21 #define RKEP_SMODE_ERR 0xffff

1.2.1.7 RK_PCIE_TASK_INFO

【说明】

定义Task信息结构体。

【定义】

```
typedef struct _RK_PCIE_TASK_INFO {
    int TaskID;
} RK_PCIE_TASK_INFO, *PRK_PCIE_TASK_INFO;
```

【成员】

成员名称	描述
TaskID	Task ID

1.2.1.8 RK_PCIE_DOWNLOAD_FIRMWARE

【说明】

定义固件下载结构体。

【定义】

```
typedef struct _RK_PCIE_DOWNLOAD_FIRMWARE {
    size_t RemoteLoadAddr;
    size_t Size;
    char Data[0];
} RK_PCIE_DOWNLOAD_FIRMWARE, *PRK_PCIE_DOWNLOAD_FIRMWARE;
```

【成员】

成员名称	描述
RemoteLoadAddr	EP设备内存地址
Size	固件长度
Data	固件内容

1.3 PCIE EP

1.3.1 rk_pcie_set_log_level

【描述】

设置日志打印等级，默认是debug。

【语法】

```
int rk_pcie_set_log_level(int level)
```

【参数】

参数名称	描述	输入/输出
level	日志等级，可选：6(verbose)、5(debug)、4(info)、3(warn)、2(error)、1(fatal)	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

1.3.2 rk_pcie_set_log_cb

【描述】

设置日志回调，可以把日志从默认console打印回调给调用者。

【语法】

```
int rk_pcie_set_log_cb(log_callback cb)
```

【参数】

参数名称	描述	输入/输出
cb	日志回调函数指针，类型为`int (*log_callback)(char *log)`	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

1.3.3 rk_pcie_device_init

【描述】

初始化PCIE设备

【语法】

```
int rk_pcie_device_init(void)
```

【参数】

无

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
typedef struct
{
    pthread_t thr_recv_rc;
    pthread_t thr_send_rc;
    char recv_rc_exit;
    char send_rc_exit;
} ep_context_st;

int main(void)
```

```
{
    int ret = 0;
    struct task_context_st task_ctx[PCIE_DMA_TASK_MAX] = {0};
    struct pcie_task_msg_st msg;
    struct task_user_cmd_st *user_cmd;

    ret = rk_pcie_device_init();
    if (ret != 0) {
        printf("pcie init device error....\n");
        return -1;
    }

    while (1) {
        ret = rk_pcie_get_msg_for_bus(&msg, 100);
        if (ret) {
            printf("task_id:%d type:%d size:%ld \n", msg.task_id, msg.type,
msg.size);
            switch (msg.type) {
            case E_TASK_MSG_CREATE:
                task_create(&task_ctx[msg.task_id], msg.task_id);
                break;
            case E_TASK_MSG_DESTROY:
                task_destroy(&task_ctx[msg.task_id], msg.task_id);
                break;
            case E_TASK_MSG_USER: {
                user_cmd = (struct task_user_cmd_st *)msg.data;
                switch (user_cmd->cmd) {
                case TASK_USER_CMD_INIT_APP:
                    app_create(&task_ctx[msg.task_id], msg.task_id);
                    break;
                case TASK_USER_CMD_DEINIT_APP:
                    app_destroy(&task_ctx[msg.task_id], msg.task_id);
                    break;
                }
            }
            break;
            default:
                printf("error msg type!!! \n");
                break;
            }
        }
    }

    rk_pcie_device_deinit();

    return 0;
}
```

【相关主题】

rk_pcie_device_deinit

1.3.4 rk_pcie_device_deinit

【描述】

反初始化pcie，释放资源

【语法】

```
int rk_pcie_device_deinit(void)
```

1.3.5 rk_pcie_task_create

【描述】

创建Task，并且初始化内存、BUFF管理等。

【语法】

```
int rk_pcie_task_create(int task_id)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID，RC端通过消息传递过来	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

请参见rk_pcie_device_init的举例。

【相关主题】

rk_pcie_device_deinit

1.3.6 rk_pcie_task_destroy

【描述】

反初始化Task，释放内存、BUFF管理等资源。

【语法】

```
int rk_pcie_task_destroy(int task_id)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

请参见rk_pcie_device_init的举例。

【相关主题】

rk_pcie_device_init

1.3.7 rk_pcie_set_ep_info

【描述】

设置EP状态信息。

【语法】

```
int rk_pcie_set_ep_info(void *status, size_t status_size)
```

【参数】

参数名称	描述	输入/输出
status	void指针，状态结构体由用户自定义	输入
status_size	状态结构体size	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

```
struct ep_info_st {
    unsigned int link_status;
    unsigned int temp;
    long long timestamp;
    unsigned char cpu_load;
    unsigned char npu_load;
    unsigned char gpu_load;
};

ep_info_st ep_info;
while(1) {
```

```
    ep_info.timestamp = time(NULL);
    rk_pcie_set_ep_info((void *)&ep_info, sizeof(struct ep_info_st));
}
```

【相关主题】

rk_pcie_get_ep_info

1.3.8 rk_pcie_get_user_cmd

【描述】

获取BAR0特定位置一块内存指针，用于用户自定义命令区。

【语法】

```
void *rk_pcie_get_user_cmd(void)
```

【参数】

无

【返回值】

返回值	描述
非NULL	正常，自定义命令区内存指针
NULL	失败

【举例】

```
struct user_cmd_st {
    unsigned int max_chn;
    unsigned int width;
    unsigned int height;
    unsigned int codecid;
    unsigned int output_id;
    unsigned int init;
};

ep_ctx.user_cmd = rk_pcie_get_user_cmd();

while(ep_ctx.user_cmd->init != 1) {
    usleep(1000);
}

ep_ctx.video_obj.width = ep_ctx.user_cmd->width;
ep_ctx.video_obj.height = ep_ctx.user_cmd->height;
ep_ctx.video_obj.input_mode = ep_ctx.user_cmd->codecid;
ep_ctx.video_obj.output_mode = ep_ctx.user_cmd->output_id;
ep_ctx.video_obj.max_chn = ep_ctx.user_cmd->max_chn;
ep_ctx.max_chn = ep_ctx.video_obj.max_chn;
```

【相关主题】

rk_pcie_get_user_cmd

1.3.9 rk_pcie_get_bar1_user_buffer

【描述】

获取bar1虚拟映射地址。

【语法】

```
void *rk_pcie_get_bar1_user_buffer(void)
```

【参数】

无。

【返回值】

返回值	描述
void *	虚拟地址
NULL	失败，请参见错误码。

1.3.10 rk_pcie_get_bar5_user_buffer

【描述】

获取bar5虚拟映射地址。

【语法】

```
void *rk_pcie_get_bar5_user_buffer(void)
```

【参数】

无。

【返回值】

返回值	描述
void *	虚拟地址
NULL	失败，请参见错误码。

1.3.11 rk_pcie_set_bar_inbound_cpu_addr

【描述】

设置bar inbound映射。

【语法】

```
u_int64_t rk_pcie_set_bar_inbound_cpu_addr(u_int32_t atu_index, u_int64_t
cpu_address, u_int32_t size)
```

【参数】

参数名称	描述	输入/输出
atu_index	atu index	输入
cpu_address	映射cpu地址	输入
size	映射大小限制	输入

【返回值】

返回值	描述
cpu_address	返回cpu地址

1.3.12 rk_pcie_get_buff

【描述】

获取可用PCIE buff。

【语法】

```
struct pcie_buff_node *rk_pcie_get_buff(int task_id, enum EBuff_Type type)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
type	PCIE Buff类型	输入

【返回值】

返回值	描述
非NULL	可用Buff指针
NULL	无可用Buff

【举例】

```
node = rk_pcie_get_buff(ctx->task_id, E_SEND);
if (node) {
    node->size = buffer_size;
    memset(node->vir_addr, index, node->size);
    add_check_info(node->vir_addr + node->size - CHECK_SIZE);
    rk_pcie_send_buff(ctx->task_id, node);
    index++;
} else {
    usleep(1000);
}
```

【相关主题】

rk_pcie_send_buff

1.3.13 rk_pcie_send_buff

【描述】

发送PCIE buff。

【语法】

```
int rk_pcie_send_buff(int task_id, struct pcie_buff_node *pbuff)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
pbuff	PCIE Buff指针	输入

【返回值】

返回值	描述
0	发送成功
非0	失败，请参见错误码

【举例】

rk_pcie_get_buff

【相关主题】

rk_pcie_get_buff

1.3.14 rk_pcie_release_buff

【描述】

释放PCIE buff。

【语法】

```
int rk_pcie_release_buff(int task_id, struct pcie_buff_node *pbuff)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
pbuff	PCIE Buff指针	输入

【返回值】

返回值	描述
0	释放成功
非0	失败，请参见错误码

【举例】

```
node = rk_pcie_get_buff(ctx->task_id, E_RECV);
if (node) {
    if (check_info(node->vir_addr + node->size - CHECK_SIZE) == 0) {
        pthread_mutex_lock(&ctx->mutex_stream_speed);
        ctx->stream_speed_index++;
        pthread_mutex_unlock(&ctx->mutex_stream_speed);
        // if (mptr != NULL) {
        //     memcpy(mptr, node->vir_addr, node->size);
        // }
        memset(node->vir_addr + node->size - CHECK_SIZE, 0, CHECK_SIZE);
    } else {
        printf("check data ==================== error \n");
        for (int i = 0; i < CHECK_SIZE; i++) {
            unsigned char *p = node->vir_addr + node->size - CHECK_SIZE;
            printf("%d ", p[i]);
        }
        printf("\ncheck data ===ssss========node->size = %d======== error \n",
(int)node->size);
    }
    rk_pcie_release_buff(ctx->task_id, node);
} else {
    usleep(1000);
}
```

1.3.15 rk_pcie_get_buff_max_size

【描述】

获取buff支持的最大size。

【语法】

```
long rk_pcie_get_buff_max_size(int task_id, enum EBuff_Type type)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
type	PCIE Buff类型	输入

【返回值】

返回值	描述
大于等于0	buff size
小于0	失败，请参见错误码

【举例】

```
long buffer_size = rk_pcie_get_buff_max_size(ctx->task_id, E_SEND);
```

1.3.16 rk_pcie_get_msg_for_bus

【描述】

等待RC端消息。

【语法】

```
int rk_pcie_get_msg_for_bus(struct pcie_task_msg_st *msg, int timeout_ms)
```

【参数】

参数名称	描述	输入/输出
timeout_ms	超时时间，-1为阻塞模式(不推荐，会一直在内核等待)，建议100ms，单位ms	输入
msg	消息结构体指针	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

请参见rk_pcie_device_init的举例。

【相关主题】

rk_pcie_device_deinit

1.3.17 rk_pcie_send_msg_to_rc_task

【描述】

发送消息到RC端

【语法】

```
int rk_pcie_send_msg_to_rc_task(int task_id, void *data, size_t size)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
data	消息指针	输入
size	消息长度	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

【举例】

请参见rk_pcie_device_init的举例。

【相关主题】

rk_pcie_device_deinit

1.3.18 rk_pcie_check_buff_avail

【描述】

确认是否有有效recv buff。

【语法】

```
int rk_pcie_check_buff_avail(int task_id)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入

【返回值】

返回值	描述
cc_true	有有效buff
cc_false	无有效buff

1.3.19 rk_pcie_get_err_event_count

【描述】

获取PCIe异常事件计数。

【语法】

```
int rk_pcie_get_err_event_count(int task_id, struct pcie_err_event_count *info)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
info	PCIe异常事件计数	输出

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码

1.3.20 rk_pcie_is_alive

【描述】

检查PCIe链路是否正常。

【语法】

```
int rk_pcie_is_alive(int task_id)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入

【返回值】

返回值	描述
cc_true	链路正常
cc_false	链路异常

1.3.21 rk_pcie_replace_buff_all

【描述】

替换所有buff的地址，用于配置用户自定义buff。

【语法】

```
int rk_pcie_replace_buff_all(int task_id, enum EBuff_Type type, struct
pcie_replace_buff_info_list_st *infos)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
type	Buff类型（E_SEND或E_RECV）	输入
infos	替换buff信息列表	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

【相关主题】

rk_pcie_replace_buff
pcie_replace_buff_info_list_st

1.3.22 rk_pcie_replace_buff

【描述】

替换单个buff的地址，用于配置用户自定义buff。

【语法】

```
int rk_pcie_replace_buff(int task_id, enum EBuff_Type type, struct
pcie_buff_node *pbuff, pcie_replace_buff_info_st *info)
```

【参数】

参数名称	描述	输入/输出
task_id	Task ID	输入
type	Buff类型（E_SEND或E_RECV）	输入
pbuff	原buff节点指针	输入
info	新buff信息	输入

【返回值】

返回值	描述
0	成功
非0	失败，请参见错误码。

【相关主题】

rk_pcie_replace_buff_all
pcie_replace_buff_info_st

1.3.23 数据类型

PCIE RC相关数据类型、数据结构定义如下：

- EBuff_Type：定义PCIE Buff类型。
- ETask_Msg_Type：定义Task消息类型。
- struct pcie_buff_node：定义PCIE buff属性结构体。
- struct pcie_task_msg_st：定义消息属性结构体。

1.3.23.1 EBuff_Type

【说明】

定义PCIE Buff类型。

【定义】

```
enum EBuff_Type {
    E_SEND = 0,
    E_RECV,
};
```

【成员】

成员名称	描述
E_SEND	发送buff。
E_RECV	接收buff。

1.3.23.2 ETask_Msg_Type

【说明】

定义Task消息类型。

【定义】

```
enum ETask_Msg_Type {
    E_TASK_MSG_NONE = 0,
    E_TASK_MSG_CREATE,
    E_TASK_MSG_DESTROY,
    E_TASK_MSG_USER,
    E_TASK_MSG_MAX,
};
```

【成员】

成员名称	描述
E_TASK_MSG_NONE	默认值，无意义
E_TASK_MSG_CREATE	创建任务
E_TASK_MSG_DESTROY	销毁任务
E_TASK_MSG_USER	用户自定义消息
E_TASK_MSG_MAX	最大消息类型

1.3.23.3 pcie_buff_node

【说明】

定义PCIE buff属性结构体。该结构体修改需要同时改RC和EP。

【定义】

```
struct pcie_buff_node{
    void *vir_addr;
    size_t phy_addr;
    size_t size;
};
```

【成员】

成员名称	描述
vir_addr	虚拟地址。
phy_addr	物理地址。
size	内存大小。 发送buff：需要用户配置数据大小，不能超过总的buff内存大小。 接收buff：对端发送过来的数据大小。

1.3.23.4 pcie_task_msg_st

【说明】

定义消息属性结构体。该结构体修改需要同时改RC和EP。

【定义】

```
struct pcie_task_msg_st {
    enum ETask_Msg_Type type;
    int task_id;
    unsigned int size;
    char lock;
    unsigned char data[TASK_MSG_DATA_SIZE];
};
```

【成员】

成员名称	描述
type	消息类型。
task_id	Task ID。
size	消息长度。
lock	保留字。
data	消息内容。

1.4 错误码

错误代码	宏定义	描述
-1	RK_PCIE_ERR_BAD	PCIE链接异常
-2	RK_PCIE_ERR_UNKNOWN	系统异常
-3	RK_PCIE_ERR_NULL_PTR	空指针错误
-4	RK_PCIE_ERR_MALLOC	分配内存失败，如系统内存不足
-5	RK_PCIE_ERR_OPEN_FILE	打开文件失败
-6	RK_PCIE_ERR_VALUE	错误异常值
-8	RK_PCIE_ERR_TIMEOUT	超时返回
-10	RK_PCIE_ERR_UNSUPPORT	不支持的功能
-13	RK_PCIE_ERR_HW_UNSUPPORT	硬件不支持
-15	RK_PCIE_ERR_MMAP	mmap失败
-65	RK_PCIE_ERR_INIT	初始化失败
-66	RK_PCIE_ERR_NOINIT	系统没有初始化
-68	RK_PCIE_ERR_NOMEM	内存不足
-69	RK_PCIE_ERR_OUTOF_RANGE	参数超出范围